

Project 2 —YOLOv5 IP Implementation Report

N26140804 張暉俊

1. What I Built

1.1 System Architecture

The skeleton of the system follows the block diagram on page 2 of the spec, but I turned it from a generic sketch into my actual design. I started from the Arm Cortex-R52 Example built into Virtualizer, reduced the core count to one, and then attached *YOLOv5_HW*, two memories, and the UARTs beside it, all hanging off the same shared bus.

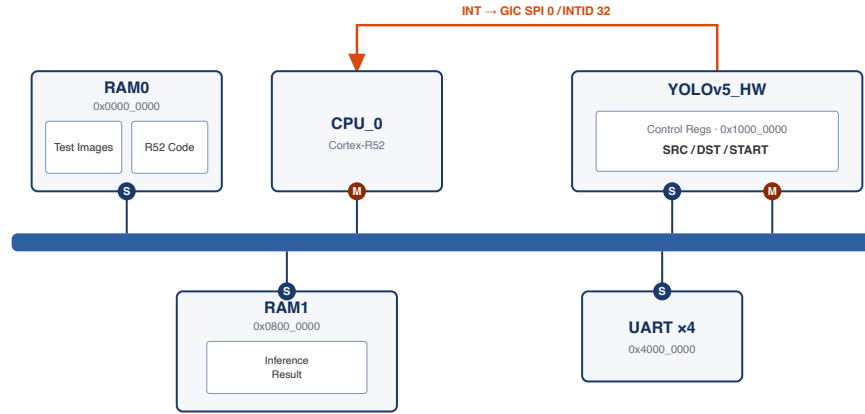


Figure 1: SoC architecture block diagram showing the R52 main core, the YOLOv5_HW coprocessor, RAM0/RAM1 and the control registers, and the INT interrupt line

Compared with the original spec diagram, I added three things. First, the control registers **SRC**, **DST**, **START** of *YOLOv5_HW*, which is a register interface I designed myself. Second, the INT interrupt line running from the IP back to the CPU. Third, the address of every block marked onto it. One thing worth spelling out is that the spec diagram draws two buses, but mine is actually a single shared bus (*SharedMemoryMap*), and besides acting as a slave that the CPU writes registers to, *YOLOv5_HW* also has its own initiator socket that can read **SRC** and write **DST** on its own. In other words it builds the DMA-style data movement into itself, so the whole diagram needs no separate DMA.

1.2 Memory Map and Control Registers

The whole SoC address space is managed by *SharedMemoryMap*. There are only three 32-bit control registers, counted from 0x10000000. **SRC** (0x10000000) holds the source address of the image to be detected, **DST** (0x10000004) holds the destination address the result is written back to, and writing 1 to **START** (0x10000008) launches one inference. Because the image container carries its own 16-byte header with the width and height inside, the IP can work out how many bytes to read by itself, which achieves what the spec calls almost the same as a DMA but with one **SIZE** controller fewer. The complete allocation is as follows.

Block	Start Address ~ End Address	Size	Purpose	Notes
<i>MEM (RAM0)</i>	0x00000000 ~ 0x01FFFFFF	32 MB	R52 local	Mainly holds code and the embedded images
<i>MEM_1 (RAM1)</i>	0x08000000 ~ 0x09FFFFFF	32 MB	Shared	Holds the inference results
<i>YOLOv5_HW</i>	0x10000000 ~ 0x1000000B	12 B	Control registers	
<i>UART_0 ~ UART_3</i>	0x40000000 ~ 0x40003FFF	4 KB each	UART registers	
<i>ARM0 (AXIS)</i>	0x50000000 ~ 0x50FFFFFF	16 MB	Core slave port	

Table 1: SoC memory map and control register allocation

1.3 Bare-metal Control Flow

After the R52 boots, *startup_r52.S* first deals with something specific to the R52. It resets into Hyp mode (EL2), so once the startup code detects Hyp I first open up access to the GIC system registers, then **eret** down to SVC, and only then does the CPU reach **main** cleanly. The main loop then runs one round for each of the three images. It writes **SRC** and **DST**, writes **START = 1** to launch the IP, and once the IP finishes reads **[count][boxes]** back from *RAM1*, then prints the coordinates of each box over the UART. After all three are done it prints **R52: 3/3 images done** to close out.

2. Training YOLOv5l and Porting It Into the System

2.1 Training

I trained YOLOv5l on the KITTI dataset, taking three traffic object classes *Car*, *Pedestrian*, *Cyclist*, at an input resolution of 640, for 100 epochs. After convergence the mAP@0.5 on the validation set reached 0.943. The trained weights were then exported to ONNX at opset 12, with input shape [1, 3, 640, 640] and output [1, 25200, 8], serving as the shared model format for the later C++ and SystemC ends.

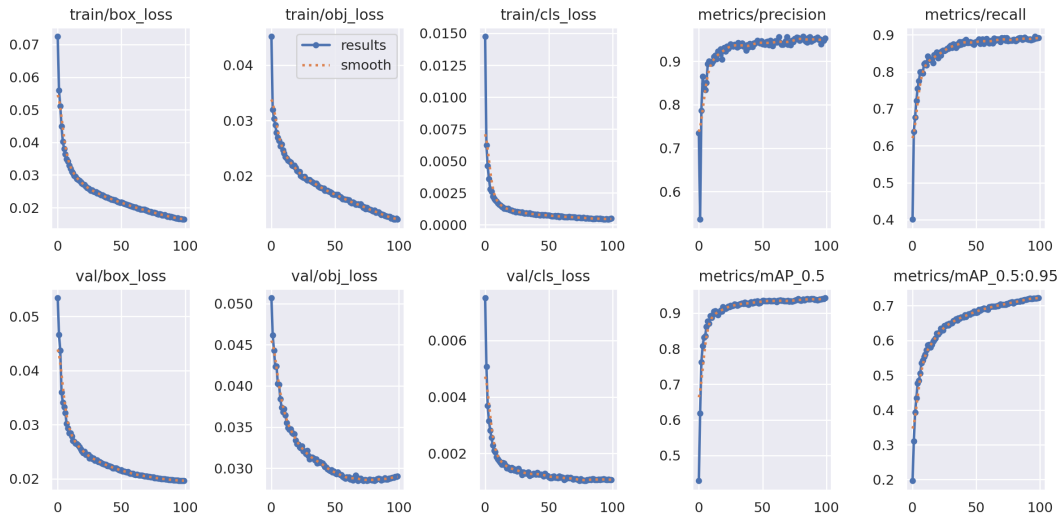


Figure 2: The loss and mAP curves over training

2.2 Porting Into the System

Porting is a road of ONNX $\rightarrow$ C++ $\rightarrow$ SystemC $\rightarrow$ Virtualizer. I first wrote a pure inference core in C++ with ONNX Runtime, and in Task B compared it against the Python reference on the same inputs, where

the relative error of the output was only 3e-6. Only after confirming that the C++ end behaved identically to the training end did I wrap this core into the *YOLOv5_HW* SystemC module, and finally import it into Virtualizer to build the TLM.

On where to place the data I made a pragmatic trade-off. I did originally intend to put the image under test into the shared *RAM1* for the IP to read, but it turned out unbearably slow. Thinking about it, having the CPU move a roughly 4MB image byte by byte into it inside the model was a bit too wishful. So for the sake of simulation efficiency, and given that every byte is a bus transaction, I decided to bake the image containers straight into the ELF so they land in *RAM0*, letting *SRC* point directly at that address, while the result is still written back to *DST* in *RAM1*.

Then, because there were no usable OpenCV dependencies in the SoC Lab, ordinary PNGs could not be decoded, and since the spec says nothing definite about the image container format, I decided to define a minimal format of my own. The design is a 16-byte little-endian header holding a magic 0x4D493559 (that is 'Y5IM'), width, height, and channels = 3, followed by $W \times H \times 3$ bytes of RGB in HWC order. The IP's *preprocess* parses this header, then hand-rolls an aspect-ratio-preserving letterbox resize to 640 (padding the gaps with 114 grey), divides by 255, and converts HWC to CHW, all without relying on OpenCV.

3. Roadblocks and How I Got Past Them

3.1 Stuck on the Build During Virtualizer Import

After importing the SystemC module, the very first build failed, and the errors all happened inside the coverage scaffolding that Virtualizer generates automatically (*covermodelBase.h*), with a rather strange message, `expected identifier before '@'`, while the module I actually wrote compiled fine. Chasing it down revealed the cause. My module's constructor took a `const char*` model-path parameter, and Virtualizer's covermodel generator emits a wrapper for string parameters that does not compile, and switching to a numeric parameter afterwards fixed it.

Right after that came the include-path problem. The ONNX Runtime headers kept not being found, and I assumed for a while that stuffing the path into Compile Flags or setting an environment variable would be enough, but neither worked. The reason is that the front end doing the syntax analysis at import time and the compiler used for the build after import are two different things. It does not read Compile Flags, nor does it honour the inherited `CPLUS_INCLUDE_PATH`, and only recognises the dedicated Include Paths field under Build > Compiler, and on top of that, `-I` must point at the directory, not at the header file itself. Even the compile stage and the link stage are two separate matters. Compiling needs ORT's include directory, while linking needs ORT's library directory plus the library name, and without the latter the build stops at `undefined reference to OrtGetApiBase`.

3.2 Fixing the Interrupt Polarity

At first I wrote the INT pin as active-LOW, carrying over the nIRQ habit from the DMA2 I designed in Project 1. But the R52's GIC only accepts rising-edge or active-HIGH level for a Shared Peripheral Interrupt, and active-LOW is reserved solely for unconfigured PPIs. Wired active-LOW, the IP would read as already asserted while idle, so the interrupt would fire spuriously at boot and instead clear on completion, which is the whole logic backwards. So I changed the IP to drive INT high on completion, and configured INTID 32 on the distributor as Group 1, active-HIGH level.

3.3 The Interrupt Never Reaching the CPU, Falling Back to Polling

Beyond that, in the original design the IP had no status register and the completion signal relied on INT alone, so the interrupt ought to be indispensable, and the ISR therefore structurally necessary. Yet running it on the platform, the CPU never once entered the ISR. I added an instrumented ELF to trace it, and only then found the real cause. That stretch of GIC distributor / redistributor MMIO is simply not decoded on the R52's CPU bus. Every read of a GIC register triggers a synchronous external abort

(DFSR 0x1210), and every write is silently dropped. By contrast the CPU-side system register interface is fine, ICC_SRE reads back 0x7, and my control register at 0x10000000 is readable too, so the problem is not in my program but in this GIC region not being attached to the bus. I had once thought 0x13090000 was confirmed working, only to find later that it was a back door poking the model directly through the Design Browser, not a real bus access.

With the demo deadline drawing near, I chose to stop at the survey I had reached rather than keep boring into the GIC. In the end I let the IP write [count][records] into DST in RAM1 on completion, where the count word itself can serve as the done flag. Before each launch the CPU seeds the count word with a sentinel 0xFFFFFFFF and then polls that location, and once the IP overwrites it with the real detection count I know it is finished, which stays visible even when the count is 0. Also, because the IP is an SC_THREAD tied to the clock edge while the R52's clock only advances when the CPU issues a bus transaction, I have it do a few extra reads of RAM1 after completion to force clock edges, so the IP sees START = 0 and re-arms. I still keep the ISR and the GIC initialisation code, so that if the platform ever does deliver INTID 32 one day, that path works too. This echoes Project 1 in an unexpected way, where the A35's nIRQ never truly arrived either and ended up falling back to polling all the same. Perhaps a signal being wired correctly at the interface does not mean it will actually be delivered —Virtualizer really is mysterious.

4. Final Results

4.1 Three Images Run Through, Results Landing in RAM1

The three test images (KITTI's 002071 / 003379 / 005872, all containing only vehicles) ran through in order, and the detection counts the R52 read back from RAM1 were 12, 4, and 3 cars respectively. Seeing this result in the memory viewer has one key point. It has to be viewed through the CPU bus, that is by doing View Memory on the ARM0 core and Goto the global DST address, rather than looking at the MEM_1 module directly, which shows up as a field of ? —a shadow store that the IP bypasses when it writes over the bus. Every box coordinate the UART prints matches the bytes in the memory viewer one for one.

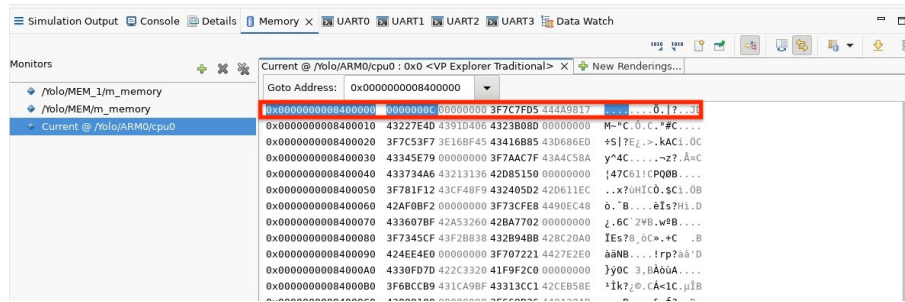


Figure 3: The count word of the first image is 12 in the RAM1 memory viewer

Take image 1 as an example. The memory viewer's Goto Address sits at 0x08400000, exactly the result start address I wrote into DST, which also falls inside the shared RAM1 region (0x08000000–0x09FFFFFF) in the memory map, and is viewed through the ARM0 core's bus perspective, so what is read is the very data the IP actually wrote back over the bus. The first word 0x0000000C is the detection count 12, matching the number of cars the UART prints. This count word is at the same time the done flag for polling. Before launch the CPU seeds it with the sentinel 0xFFFFFFFF, and only when the IP overwrites it with the real number does this round of inference count as finished.

After the count, each box occupies six consecutive 32-bit words, in order the class, the confidence, the x and y of origin, and the width and height of the box. Taking the first box to check, the 0x00000000 at 0x08400004 is class 0, that is Car, the 0x3F7C7FD5 at 0x08400008 decodes as 0.986 under IEEE-754, the 0x444A9817 at 0x0840000C is 810.3, and then the three words at 0x08400010, 0x08400014, 0x08400018

are 162.5, 291.7, and 163.7. Truncating the decimals to integers gives exactly the first UART line [Car] 98% origin(810,162) size(291x163), so the bytes in memory are indeed just as printed on the UART.

```

Simulation Output Console Details Memory UART0 X
GIC init: wake redistributor write
GIC init: wake redistributor done
GIC init: write GICD_CTLR
GIC init: GICD_CTLR done
GIC init: read ICC_SRE
  ICC_SRE before = 0x00000007
GIC init: write ICC_SRE
  ICC_SRE after = 0x00000007
GIC init: write ICC_PMR
GIC init: write ICC_BPR1
GIC init: write ICC_IGRPEN1
GIC init: done
GIC enable: skip per-SPI MMIO for INTID 32
GIC enable: using Virtualizer Interrupt Table mapping
Run image 1
  SRC = 0x000010D8
  DST = 0x08400000
Image 1: 12 detections
[Car] 98% origin(810,162) size(291x163)
[Car] 98% origin(0,193) size(429x180)
[Car] 97% origin(329,183) size(161x108)
[Car] 96% origin(414,164) size(107x87)
[Car] 95% origin(1159,182) size(82x93)
[Car] 95% origin(485,171) size(70x51)
[Car] 93% origin(671,176) size(43x31)
[Car] 92% origin(156,177) size(103x34)
[Car] 90% origin(552,173) size(25x21)
[Car] 78% origin(0,179) size(204x79)
[Car] 71% origin(266,173) size(83x35)
[Car] 69% origin(359,166) size(76x46)

```

Figure 4: UART0 printing the 12 box coordinates read back for image 1

4.2 The Detections Are Correct

To make sure these numbers are not merely correct in count but that the boxes genuinely land on the cars, I drew the detection boxes back onto the original images. Taking image 002071 as an example, the vehicles are all boxed correctly and the confidences are reasonable, which shows that the whole porting path from training through ONNX and C++ to SystemC did not distort anywhere along the way.

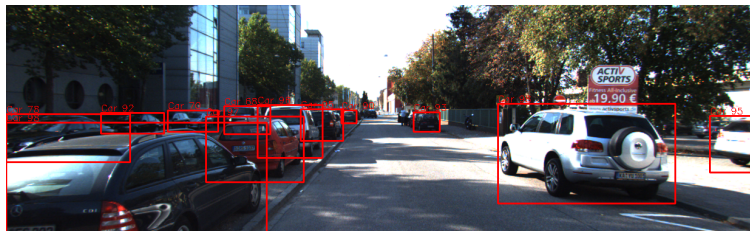


Figure 5: Detection result of test image 002071, 12 cars

Closing Thoughts

What this project most resembles about Project 1 is that the problem often lies not in the code I wrote but in the gaps among the spec, the manuals, and the tools. The covermodel failing to compile over a single string parameter, the include path having to go into a specific field, the entire GIC not being attached to the bus—none of these are visible from the surface of the code. They have to be pieced together, or rather dug out, slowly from error messages, the console, and the fine print of the manuals.